

Centro de Procesamiento de Datos



UNIVERSIDAD
DE GRANADA

Práctica 1. Contenedores Docker

Juan Navarro Maldonado

Compartiendo el directorio host con el ordenador.....	4
Creación interactiva de un contenedor docker.....	5
Acceso por SSHFS.....	6
Crear un volumen utilizando un controlador de volumen.....	7

Compartiendo el directorio host con el ordenador

La idea principal de este apartado es compartir un directorio local del host con un contenedor Docker, para que los archivos almacenados en nuestra máquina local puedan ser accesibles y utilizados dentro del contenedor. Específicamente en este apartado, nos vamos a centrar en la creación de un contenedor **NGINX**, donde montamos una carpeta local que contiene un archivo `index.html` personalizado y el archivo de configuración de NGINX.

Este proceso se logra utilizando la opción **-v** en el comando **docker run**, lo que permite que los cambios realizados en los archivos locales se reflejen automáticamente en el contenedor, sin necesidad de modificar el contenedor cada vez que hay un cambio.

Para ello ejecutaremos el siguiente comando:

```
Unset
docker run --name nginx2 -v /ruta/a/tu/carpeta:/usr/share/nginx/html -v
/ruta/a/tu/carpeta:/usr/share/nginx/html -p 8081:80 -p 8444:443 -d nginx
```

También tenemos que tener en cuenta que tenemos que mapear los puertos del contenedor a nuestra máquina local para poder acceder al contenedor. Estos puertos en cuestión son **HTTP** y **HTTPS** respectivamente.

Para el proceso de montar la carpeta, deberemos de crear un nuevo directorio que contenga la siguiente información:

```
~/Documents/github/CPD/Practical (0.038s)
Tree .
├── conf
│   ├── conf.d
│   ├── fastcgi.conf
│   ├── fastcgi_params
│   ├── koi-utf
│   ├── koi-win
│   ├── mime.types
│   ├── modules-available
│   ├── modules-enabled
│   │   ├── 50-mod-http-geoip2.conf -> /usr/share/nginx/modules-available/mod-http-geoip2.conf
│   │   ├── 50-mod-http-image-filter.conf -> /usr/share/nginx/modules-available/mod-http-image-filter.conf
│   │   ├── 50-mod-http-xslt-filter.conf -> /usr/share/nginx/modules-available/mod-http-xslt-filter.conf
│   │   ├── 50-mod-mail.conf -> /usr/share/nginx/modules-available/mod-mail.conf
│   │   ├── 50-mod-stream.conf -> /usr/share/nginx/modules-available/mod-stream.conf
│   │   └── 70-mod-stream-geoip2.conf -> /usr/share/nginx/modules-available/mod-stream-geoip2.conf
│   ├── nginx.conf
│   ├── proxy_params
│   ├── scgi_params
│   ├── sites-available
│   │   └── default
│   ├── sites-enabled
│   │   └── default -> /etc/nginx/sites-available/default
│   ├── snippets
│   │   ├── fastcgi-php.conf
│   │   └── snakeoil.conf
│   ├── uwsgi_params
│   └── win-utf
└── html
    └── index.html

9 directories, 21 files
```

Ilustración 1

Estos serían los ficheros de configuración por defecto de NGINX más un fichero HTML personalizado para poder visualizar los resultados.

Una vez tenemos el directorio montado con los ficheros necesarios, procedemos a ejecutar el código:

```
~/Documents/github/CPD/Practica1 (0.336s)
docker run --name nginx1 -v ~/Documents/github/CPD/Practica1/html:/usr/share/nginx/html:ro -v ~/Documents/github/CPD/Practica1/conf:/etc/nginx:ro -p 8081:80 -p8444:443 -d nginx
92d71a952713e93d38d4daf56e10bbdf4be895f8e2204d75eacd19c4ed997c72

~/Documents/github/CPD/Practica1 (0.098s)
docker ps

```

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS	NAMES
92d71a952713	nginx	"/docker-entrypoint..."	7 seconds ago	Up 6 seconds	0.0.0.0:8081->80/tcp, 0.0.0.0:8444->443/tcp	nginx1

```
~/Documents/github/CPD/Practica1
|
```

Ilustración 2

El resultado sería el siguiente:

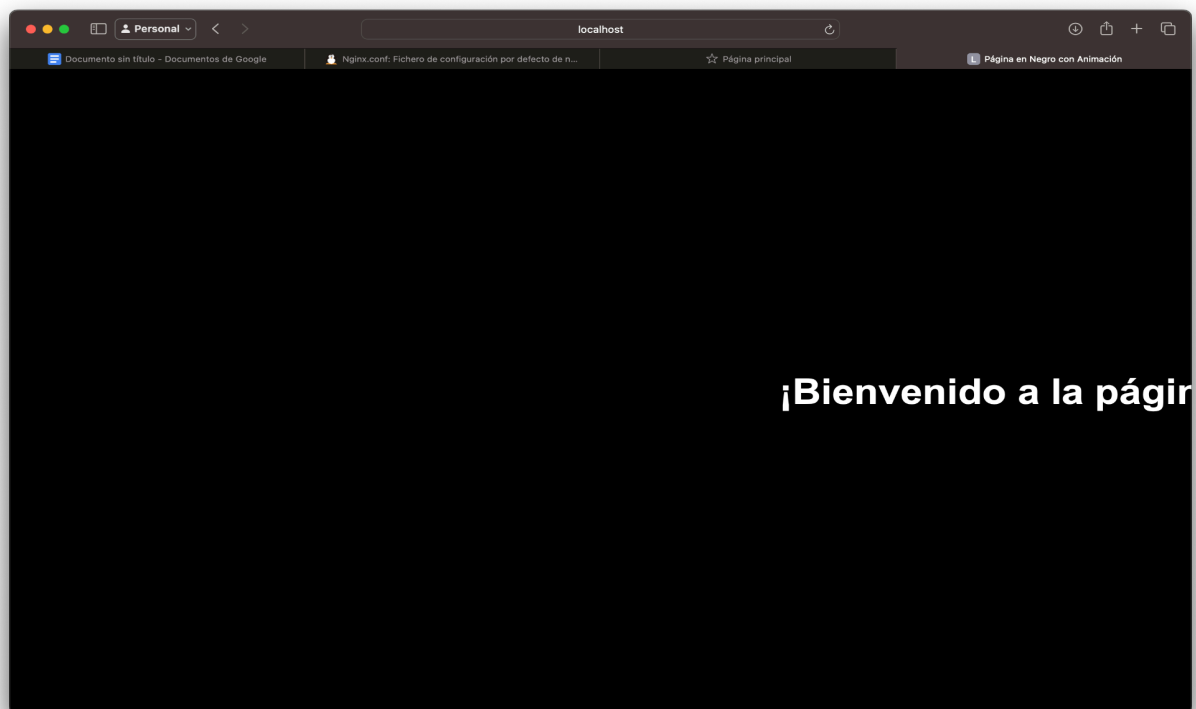


Ilustración 3

Creación interactiva de un contenedor docker

Un **contenedor** interactivo en Docker es un contenedor que se ejecuta de manera que puedas interactuar directamente con él a través de una terminal, permitiéndote ejecutar comandos en tiempo real dentro del contenedor. Esto se logra utilizando la opción **-it** en el comando **docker run**, que habilita la interacción a través de una consola.

Para crear un contenedor de forma interactiva, como ejemplo, crearemos un contenedor de **ubuntu** y lanzaremos la terminal con **bash**:

Unset

```
docker run -it ubuntu bash
```

Podemos verificar que estamos dentro del contenedor con el comando **uname -a**

```
~/Documents/github/CPD/Practica1
docker run -it ubuntu bash
Unable to find image 'ubuntu:latest' locally
latest: Pulling from library/ubuntu
dafa2b0c44d2: Pull complete
Digest: sha256:dfc10878be8d8fc9c61cbff33166cb1d1fe44391539243703c72766894fa834a
Status: Downloaded newer image for ubuntu:latest
root@7751e147654b:/# uname -a
Linux 7751e147654b 6.6.22-linuxkit #1 SMP PREEMPT_DYNAMIC Fri Mar 29 12:23:08 UTC 2024 x86_64 x86_64 x86_64 GNU/Linux
root@7751e147654b:/#
```

Ilustración 4

Acceso por SSHFS

Lo primero que debemos realizar es un contenedor local basado en **Alpine** con privilegios. Para ello vamos a ejecutar la siguiente orden:

Unset

```
docker run -it --privileged=true alpine
```

Para posteriormente añadir el paquete **sshfs**:

Unset

```
apk add sshfs
```

Ahora procederemos a las pruebas creando un directorio llamado **mi_turing** para posteriormente, con el comando siguiente, montar el directorio remoto en local:

Unset

```
/ # sshfs juannavarrom@turing.ugr.es:. mi_turing
```

Cabe destacar que SSHFS es una **herramienta** utilizada para montar un sistema de archivos remoto en un servidor a través de SSH, lo que permite acceder a los archivos remotos como si estuvieran en nuestro sistema local, por lo que con este comando hemos montado el directorio remoto de mi usuario '**juannavarrom**' que está alojado en el servidor de **turing.ugr.es** en el directorio local que he llamado **mi_turing**.

En la siguiente ilustración podemos ver el resultado y los ficheros que tengo alojados:



El primer paso es instalar el plugin **vieux/sshfs** con el comando:

Esto instalará el plugin para Docker, permitiendo crear volúmenes montados a través de SSH.

Lo siguiente es verificar la instalación con el siguiente comando:

Si todo ha salido correctamente debería salirnos el siguiente resultado:



Por último, debemos crear el volumen utilizando [vieux/sshfs](#) en un directorio remoto que esté disponible vía SSH como, por ejemplo, mi directorio en [turing.ugr.es](#). Esto lo realizaremos con el siguiente comando:

```
Unset
docker volume create \
--driver vieux/sshfs \
-o sshcmd=juannavarrom@turing.ugr.es \
-o password=mi_contraseña \
ssh_volume
```

Por último, creamos un contenedor interactivo de Ubuntu donde montaremos el volumen creado anteriormente:

```
Unset
docker run -it --name ubuntu_sshfs \
-v ssh_volume:/mnt/remote_data \
ubuntu /bin/bash
```